TEST TECHNIQUE DEVOPS • 13 OCTOBRE 2026

Tython

Rapport technique et état d’avancement

CI/CD · Docker · Proxmox · Observabilité

Auteur : Aymane — compte GitHub **Aymane-Elb**
Périmètre : application React / Express / PostgreSQL et chaîne de livraison associée.

Synthèse

L’application et sa conteneurisation ont été réalisées et vérifiées localement. Le pipeline CI et le déploiement par SSH avec Docker Compose sont préparés. Proxmox est installé ; l’installation de la VM Debian cible est encore en cours. La publication des images et le fonctionnement sur cette cible restent à démontrer.

Résultats établis

- Application de gestion de tâches fonctionnelle, API Express et stockage PostgreSQL persistant.
- Construction des images frontend et backend ; quatre tests d’intégration backend réussis lors des vérifications locales.
- Configuration de production testée dans une instance Docker isolée : trois services sains et données conservées après redéploiement.
- Laboratoire Proxmox accessible et VM Debian 100 créée.

Réserves de livraison

Les fichiers de production ne sont pas encore commités sur la branche **feature/deployment**. Aucun résultat distant de GitHub Actions, téléchargement GHCR ou déploiement applicatif dans Debian n’est confirmé par les éléments disponibles.

Nature du document. Ce rapport décrit un état d’avancement, et non une attestation de mise en production complète. Il distingue la présence des fichiers, leur validation locale et les preuves distantes restant à recueillir.

TYTHON / RAPPORT TECHNIQUE

1. Exigences et couverture

Référence : « Test Technique DevOps.pdf » fourni pour l'exercice.

L'objectif est de mettre en place une chaîne reproductible : gestion du code, contrôles, packaging, déploiement et observabilité. Le développement fonctionnel reste volontairement limité à une application simple.

Exigence	Réalisation	État
React, Express, PostgreSQL	Liste de tâches et API avec persistance.	**Validé localement**
GitHub principal / GitLab miroir	Remotes configurés ; stratégie de synchronisation manuelle après fusion.	**En place** Synchronisation finale à vérifier
Branches et CONTRIBUTING.md	main, dev et feature/* ; règles dans contribution.md.	**Partiel** Nom du fichier à corriger
Docker et Compose local	Deux Dockerfiles ; image PostgreSQL officielle ; compose.yaml.	**Validé localement**
CI GitHub Actions	Lint backend, tests PostgreSQL, build React et images Docker.	**Implémenté** Preuve CI distante manquante
Publication des images	GHCR sur push main ; tags latest et sha-<commit>.	**Configuré** Publication à confirmer
Déploiement sur VM / LXC	Option A : SSH + Compose. Proxmox installé, Debian en installation.	**En cours**
Secrets et sécurité minimale	Variables injectées, exemples sans secret réel, ports limités en production.	**Implémenté**
Observabilité minimale	Healthchecks, restart et rotation des logs.	**Validé localement**
Documentation et preuves	README et DEPLOYMENT.md ; preuves locales consignées.	**À finaliser**

Options retenues et bonus

Une seule méthode de déploiement est nécessaire : SSH + Compose a été retenue. Ansible et Cloud-init ne sont donc pas requis en complément. L'observabilité est couverte par les logs avec rotation et les contrôles de santé ; Prometheus/Grafana ne sont pas indispensables pour cette option.

Bonus non réalisés : CD automatique, staging séparé, GitLab CI, HTTPS, rate limiting et en-têtes de sécurité supplémentaires. Nginx est présent comme reverse proxy. Les tags d'images par commit sont configurés ; aucune release Git v1.0.0 n'est confirmée.

TYTHON / RAPPORT TECHNIQUE

2. Application et intégration continue

Architecture applicative



React fournit une interface en français pour ajouter, terminer, filtrer et supprimer des tâches. Express expose les routes /api/tasks et /api/health. Ce dernier contrôle aussi la connexion à PostgreSQL. Les requêtes SQL utilisent des paramètres et les titres sont validés côté API.

Élément	Choix technique
Frontend	Build React/Vite sous Node 22 ; fichiers compilés servis par Nginx. Le proxy /api conserve le même point d'accès pour le navigateur.
Backend	Express, pilote pg, dépendances de production et utilisateur node non root. Le schéma SQL est initialisé au démarrage.
Base de données	Image officielle postgres:17-alpine ; volume persistant. Aucun Dockerfile PostgreSQL personnalisé n'est nécessaire ici.

Git et stratégie de livraison

GitHub est la source principale. Les branches feature/* sont fusionnées dans dev par Pull Request, puis dev dans main pour une livraison stable. GitLab reçoit les branches mises à jour par push. Cette synchronisation est manuelle ; les fusions sont centralisées sur GitHub.

À la dernière lecture des références Git locales, dev, origin/dev et gitlab/dev pointent sur le commit **5f1a416**, qui intègre la PR de dockerisation. main reste au commit initial connu **f7420d3**. Il ne s'agit pas d'une interrogation en temps réel des plateformes.

Workflow .github/workflows/ci.yml

- Déclenchement : Pull Request vers dev ou main ; push vers main.
- Contrôles : npm ci, ESLint backend, quatre tests d'intégration avec une base PostgreSQL temporaire, puis compilation React.
- Après réussite : construction des images frontend et backend dans deux jobs de matrice.
- Publication GHCR uniquement sur push main, via GITHUB_TOKEN et la permission packages: write ; tags latest et sha du commit.

Le backend JavaScript n'exige pas de compilation distincte : sa construction Docker constitue l'étape de packaging. Lint frontend et tests frontend ne sont pas inclus ; le sujet autorise un périmètre backend pour ces contrôles.

TYTHON / RAPPORT TECHNIQUE

3. Déploiement, sécurité et exploitation

Déploiement reproductible préparé

docker-compose.prod.yml utilise des images de registre et un volume de données distinct du développement. Le script deploy.sh valide la configuration, télécharge les images, recrée les services, attend leurs healthchecks et interroge /api/health via Nginx. Les données sont conservées pendant la recréation.

Le script s'exécute manuellement sur le serveur après connexion SSH. Il ne provisionne pas Proxmox et n'est pas déclenché automatiquement par la CI. Une mise à jour peut provoquer une courte interruption ; aucun rollback automatique n'est implémenté.

Domaine	Mesures mises en place
Configuration et secrets	deploy/.env à créer sur le serveur depuis deploy/.env.example. Fichiers réels ignorés par Git et exclus des images. Aucun secret applicatif dans les Dockerfiles.
Connexion PostgreSQL	En production : PGHOST, PGPORT, PGUSER, PGPASSWORD et PGDATABASE. DATABASE_URL reste disponible pour le développement.
Exposition réseau	Seul le frontend publie un port. Express et PostgreSQL restent accessibles par les réseaux Docker. Le réseau de la base est interne.
Accès à l'application	Écoute locale par défaut ; accès distant documenté par tunnel SSH ou IP privée. Pas d'authentification applicative ni de TLS livré à ce stade.
Exécution	Backend non root ; no-new-privileges pour les conteneurs applicatifs ; restart: unless-stopped sur les trois services.
Observabilité	Healthchecks PostgreSQL, API et proxy ; logs json-file limités à 10 Mo × 3 fichiers par conteneur.
Sauvegarde et retour arrière	Procédure pg_dump et retour à un ancien tag d'image documentés. Le rollback applicatif ne restaure pas automatiquement les données.

Point requis avant publication

L'adaptation du backend aux variables PG* fait partie des changements locaux non commités. Elle doit être intégrée aux images publiées avant d'utiliser le Compose de production.

Limites : un état unhealthy ne redémarre pas à lui seul le conteneur. La surveillance actuelle ne comprend ni tableau de bord de métriques ni alertes. Le déploiement vise un laboratoire privé ; une exposition publique nécessite des mesures supplémentaires.

TYTHON / RAPPORT TECHNIQUE

4. Infrastructure et vérifications

État réel du laboratoire

Arch Linux → QEMU/KVM avec virt-manager → Proxmox VE → VM Debian → Docker. Docker dans la VM Debian représente la cible de déploiement à terminer.

Composant	État observé
Hôte de laboratoire	Ordinateur personnel conservé ; Proxmox installé dans un disque virtuel sous KVM.
Proxmox VE	Installation effective ; interface web accessible ; connexion administrateur établie. Les captures montrent Proxmox VE 9.2.2.
VM 100 — tython-server	Créée avec 2 vCPU, 2 Gio de mémoire et un disque virtuel de 24 Gio. ISO Debian importée et démarrage de l'installateur observé.
Installation Debian	Partitionnement et système de base effectués. Dernier écran observé : configuration du miroir et du proxy HTTP du gestionnaire de paquets.
Cible applicative	Fin de l'installation, démarrage autonome, SSH vers Debian, installation Docker et déploiement applicatif non encore confirmés.

Vérifications locales déjà réalisées

Contrôle	Résultat et portée
Qualité / tests / build	ESLint backend réussi, quatre tests d'intégration PostgreSQL réussis et compilation React réussie lors des vérifications locales.
Images Docker	Images frontend et backend construites avec succès. Configuration Nginx incluse dans l'image finale.
Script de production	Exécuté avec des images locales dans un projet Docker isolé ; les trois services ont atteint l'état healthy.
Persistance	Tâche créée, conteneurs recréés, tâche relue et modifiée avec succès. Tâche et instance de test supprimées ensuite.

Preuves encore nécessaires

Exécution GitHub Actions verte ; deux images visibles dans GHCR ; récupération de ces images depuis Debian ; application accessible sur la VM ; persistance et états de santé sur cette cible. Les vérifications locales ne remplacent pas ces preuves distantes.

Les observations d'infrastructure proviennent des captures de l'installation. Ces captures ne sont pas jointes au présent PDF ; elles devront être regroupées avec les preuves finales de livraison.

TYTHON / RAPPORT TECHNIQUE

5. Livrables et plan de finalisation

Livrable	Situation dans le dossier de travail
client/Dockerfile, server/Dockerfile client/nginx.conf, compose.yaml	Présents et suivis par Git. compose.yaml remplit le rôle du docker-compose.yml demandé.
.github/workflows/ci.yml eslint.config.mjs	Présents et suivis par Git ; résultats distants à confirmer.
contribution.md	Règles rédigées ; renommer en CONTRIBUTING.md pour suivre le sujet.
README.md et .env.example	Présents ; procédure locale à aligner et variable DOCKER_DATABASE_URL à ajouter au modèle racine.
docker-compose.prod.yml deploy.sh, deploy/.env.example DEPLOYMENT.md	Créés localement mais non suivis par Git au moment du constat. À commiter sur feature/deployment.

Actions prioritaires

1. Terminer Debian. Installer SSH et les utilitaires usuels, finir le chargeur de démarrage, redémarrer sur disque et vérifier réseau, DNS et accès SSH.

2. Corriger la livraison. Renommer contribution.md, compléter le modèle d'environnement et distinguer dans le README les modes développement et Docker. Aujourd'hui db:up démarre tous les services, ce qui peut entrer en conflit avec npm run dev.

3. Versionner le déploiement. Inclure les fichiers nouveaux et la modification PG* du backend, puis ouvrir une PR vers dev et obtenir les contrôles verts.

4. Publier. Fusionner dev vers main après validation et vérifier les deux images GHCR, en conservant le tag du commit retenu.

5. Déployer dans Debian. Installer Docker/Compose, configurer les secrets hors Git, récupérer les fichiers et exécuter deploy.sh.

6. Valider et remettre. Vérifier API, interface, healthchecks et persistance ; synchroniser GitLab et joindre les captures. Une release Git versionnée peut compléter la livraison en bonus.

Références et périmètre du constat

Sujet : Test Technique DevOps.pdf. Sources techniques : fichiers du dépôt, historique Git local, vérifications locales déjà exécutées et captures de l'installation. Aucun audit exhaustif de l'historique des secrets ni nouvelle exécution distante du pipeline n'est revendiqué.

GitHub : github.com/Aymane-Elb/tython
GitLab : gitlab.com/Aymane-Elb/tython
Branche de travail constatée : feature/deployment.